

FitBuddy: AI-Powered Fitness Plan Generator

Team ID	SWTID-2026-5841
Team Leader	Dhanush M
Team Members	Jayavardhani P, Sandhiya S, V Brindha, Ishwarya

Project Description:

FitBuddy harnesses Generative AI to give users a fast, personalised workout-planning experience. The platform includes a Workout Plan Generator, which creates a structured 7-day plan from the user's age, weight, goal and preferred intensity; a Feedback Updater, which revises the complete plan from the user's comments; and a Nutrition Tip Generator, which returns a short, balanced tip for the chosen goal.

Using the Google Gemini API (gemini-2.5-flash), FitBuddy processes validated user inputs to deliver clear, safety-focused fitness guidance. Built with FastAPI and SQLite and packaged with Docker, the platform offers a simple, reliable experience. With secure API key management through a .env file and careful input validation, FitBuddy helps beginners and casual exercisers start and continue a healthy routine with confidence.

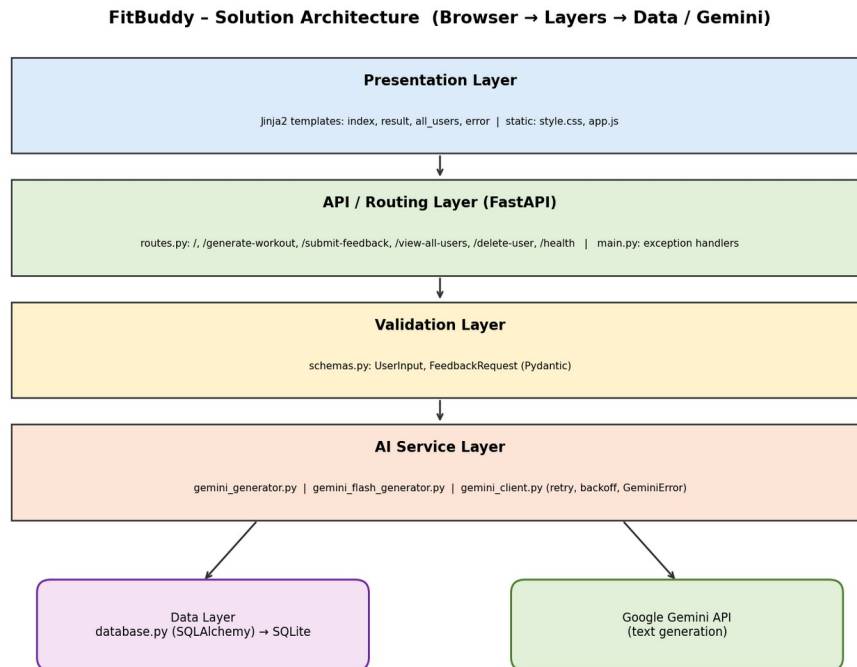
Scenarios

Scenario 1: A beginner enters age 21, weight 60 kg, goal "general wellness" and intensity "low". FitBuddy returns a gentle 7-day plan with warm-ups, light workouts, rest days and a simple hydration and sleep tip.

Scenario 2: A gym-goer chooses "muscle gain" with "high" intensity. The plan focuses on strength sessions with gradual progression, then the user asks for fewer leg days; FitBuddy returns a complete revised plan.

Scenario 3: A user selects "weight loss" with "medium" intensity. FitBuddy builds a cardio-focused plan and a non-restrictive nutrition tip, while reminding the user to consult a professional if they have pain or medical concerns.

Technical Architecture:



Description: FitBuddy uses a layered architecture. The frontend provides HTML pages rendered with Jinja2 and styled with CSS and JavaScript. The FastAPI backend validates data with Pydantic, builds prompts and calls the Google Gemini API through a shared client with retry and backoff. Users and plans are stored in an SQLite database through SQLAlchemy. The application runs locally with Uvicorn or in a Docker container.

ER Diagram:

FitBuddy - ER Diagram (single entity)

users		
id	INTEGER	PK
user_id	VARCHAR(100)	UNIQUE, NOT NULL
username	VARCHAR(100)	NOT NULL
age	INTEGER	NOT NULL
weight	FLOAT (kg)	NOT NULL
goal	VARCHAR(100)	NOT NULL
intensity	VARCHAR(20)	NOT NULL
original_plan	TEXT	NULL
updated_plan	TEXT	NULL
nutrition_tip	TEXT	NULL

Description: FitBuddy stores data in one table, users. Each row holds a user's unique user_id, name, age, weight, goal, intensity, the original generated plan, the latest updated plan and the nutrition tip. Re-using a user_id resets that record with a newly generated plan.

Pre-requisites:

- **FastAPI Knowledge:** [FastAPI Documentation](#)
- **Google Gemini API Familiarity:** [Gemini API Documentation](#)
- **HTML, CSS and JavaScript Skills:** [W3Schools Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **SQLAlchemy and SQLite:** [SQLAlchemy Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Docker (optional):** [Docker Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Gemini model for workout planning.
- **Activity 1.3:** Define the architecture of the application.
- **Activity 1.4:** Set up the development environment and install dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the Workout Plan Generator, Feedback Updater and Nutrition Tip Generator.
- **Activity 2.2:** Implement the FastAPI backend for routing, validation and database access.

Milestone 3: Application Development

- **Activity 3.1:** Write the routes, schemas, database model and Gemini client.

Milestone 4: Frontend Development

- **Activity 4.1:** Design the user interface with HTML, CSS and JavaScript.
- **Activity 4.2:** Create dynamic Jinja2 templates to show plans, tips and errors.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Run the application locally and test with pytest.
- **Activity 5.2:** Package and run the application with Docker.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone we select the Gemini model used to generate plans and define how the frontend, backend, AI service and database work together.

Activity 1.1: Generate a Google Gemini API Key

Step 1 — Sign in: Open Google AI Studio at <https://aistudio.google.com> and sign in with a Google account.

Step 2 — Open API keys: Choose "Get API key" from the menu.

Step 3 — Create the key: Click "Create API key", select or create a project, and copy the key immediately.

[Insert screenshot: Google AI Studio API keys page]

Step 4 — Add the key to the project: Create a .env file in the project root (copy .env.example) and set the values:

```
GOOGLE_API_KEY=your_google_api_key_here
GEMINI_WORKOUT_MODEL=gemini-2.5-flash
GEMINI_TIP_MODEL=gemini-2.5-flash
ADMIN_USERNAME=admin
ADMIN_PASSWORD=choose-a-strong-password
```

Important — never commit your API key: the .gitignore file excludes .env. Share only .env.example, and create a new key immediately if a real key is ever exposed.

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand the requirements:** FitBuddy needs clear, structured plain-text plans and short tips for a general audience.
- **Explore the Gemini documentation:** compare available models by speed, quality, cost and limits.
- **Select the model:** gemini-2.5-flash for a balance of quality and speed; temperature 0.7 for plans and revisions, 0.5 for the more consistent nutrition tip, with a 60-second timeout. Model names are configurable through environment variables.

Activity 1.3: Define the Architecture of the Application

- **Frontend:** users enter details, choose goal and intensity, read the plan and submit feedback.
- **Backend:** FastAPI routes validate input with Pydantic, call the AI services and save data.
- **AI integration:** prompts include safety rules and ask for plain text; a shared client retries transient errors.
- **Data:** SQLAlchemy model User stored in SQLite.

Activity 1.4: Set Up the Development Environment

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

```
# requirements.txt
fastapi>=0.115
uvicorn[standard]>=0.30
jinja2>=3.1
sqlalchemy>=2.0
python-multipart>=0.0.9
google-genai>=1.0
python-dotenv>=1.0
```

Project structure: app/ (main.py, routes.py, schemas.py, database.py, gemini_client.py, gemini_generator.py, gemini_flash_generator.py), templates/, static/, tests/, run.py, Dockerfile, requirements.txt, .env.example, README.md.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Features

- **Workout Plan Generator:** creates Day 1 to Day 7 with a 5 to 10 minute warm-up, main workout with exercises and sets/reps or duration, rest and a cooldown, matched to age, weight, goal and intensity.
- **Feedback Updater:** takes the latest plan and the user's feedback and returns a complete revised 7-day plan; repeated feedback builds on the newest version.

- **Nutrition Tip Generator:** returns a 2 to 4 sentence tip about ordinary food, hydration, sleep or recovery, without restrictive calorie targets or supplements.

Activity 2.2: Implement the FastAPI Backend

- **Define routes:** home page, plan generation, feedback, admin list, delete user and health check.
- **Process user input:** a form in index.html posts to /generate-workout; Pydantic validates each field.
- **Integrate API calls:** the route calls generate_workout_gemini() and generate_nutrition_tip_with_flash(), then saves the result with upsert_user().

Validation rules (schemas.py):

```
class UserInput(BaseModel):
    username: str = Field(min_length=1, max_length=100)
    user_id: str = Field(min_length=1, max_length=100, pattern=USER_ID_PATTERN)
    age: int = Field(ge=13, le=100)
    weight: float = Field(gt=0, le=500)
    goal: Literal["general wellness", "muscle gain", "weight loss",
                  "flexibility", "cardio fitness"]
    intensity: Literal["low", "medium", "high"]
```

Milestone 3: Application Development

Activity 3.1: Writing the Main Application Logic

Core routes (routes.py):

```
@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(request, "index.html")

@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(request: Request, data: Annotated[UserInput, Form()]):
    plan = generate_workout_gemini(data.username, data.age, data.weight,
                                   data.goal, data.intensity)

    try:
        tip = generate_nutrition_tip_with_flash(data.goal)
    except GeminiError:
        logger.exception("Nutrition tip generation failed; saving plan without a tip")
        tip = None

    user = upsert_user(data.model_dump(), plan, tip)
    return _render_result(request, user, plan)

@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback(request: Request, data: Annotated[FeedbackRequest, Form()]):
    user = get_user(data.user_id)
    if not user or not user.original_plan:
        raise HTTPException(404, "User ID not found. Please generate a plan first.")
    current = user.updated_plan or user.original_plan
    revised = update_workout_plan(current, data.feedback)
    save_updated_plan(user.user_id, revised)
    return _render_result(request, user, revised,
                          "Your workout plan has been updated using your feedback.")
```

Gemini client with retry and backoff (gemini_client.py):

```
def generate_text(model: str, prompt: str, temperature: float) -> str:
    """Call Gemini with retry and exponential backoff on transient errors."""
    last_error = None
```

```
for attempt in range(1, MAX_ATTEMPTS + 1):
    try:
        response = _get_client().models.generate_content(
            model=model, contents=prompt,
            config=types.GenerateContentConfig(temperature=temperature))
    except errors.APIError as exc:
        last_error = exc
        if exc.code not in RETRYABLE_CODES or attempt == MAX_ATTEMPTS:
            break
        time.sleep(2 ** (attempt - 1))
        continue
    text = (response.text or "").strip()
    if not text:
        raise GeminiError("Gemini returned an empty response.")
    return text
raise GeminiError("Gemini request failed.") from last_error
```

Database model (database.py):

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(100), unique=True, index=True, nullable=False)
    username = Column(String(100), nullable=False)
    age = Column(Integer, nullable=False)
    weight = Column(Float, nullable=False)
    goal = Column(String(100), nullable=False)
    intensity = Column(String(20), nullable=False)
    original_plan = Column(Text, nullable=True)
    updated_plan = Column(Text, nullable=True)
    nutrition_tip = Column(Text, nullable=True)
```

Prompt design

The workout prompt sets the role ("FitBuddy, a responsible fitness planning assistant"), lists the user's details, and requires Day 1 to Day 7, a warm-up, main workout, cooldown, gradual progression and plain-text output. It forbids extreme exercise, starvation, dehydration and supplements, and tells the user to consult a professional for pain, injury or illness. In the feedback prompt, the user's text is placed between markers and treated as data, not instructions.

Milestone 4: Frontend Development

Activity 4.1: Designing and Developing the User Interface

- **Base HTML:** templates/index.html holds the input form; result.html shows the plan, tip and feedback box; all_users.html lists users; error.html shows friendly errors.
- **Styling:** static/style.css provides a clean, responsive layout for phones and computers.
- **Form fields:** name, user ID, age, weight, goal (five options) and intensity (low, medium, high).

Activity 4.2: Dynamic Content Rendering

- **Jinja2 templates:** the server renders the plan, tip and messages with values passed from `_render_result()`.
- **JavaScript:** static/app.js shows a loading overlay while the AI is working and restores the page when the user navigates back.

Milestone 5: Testing and Deployment

Activity 5.1: Local Testing and Verification

```
python run.py
# open http://127.0.0.1:8000
pip install -r requirements-dev.txt
pytest -q
```

Try all five goals and three intensities, submit feedback more than once, and try invalid input (age 12, a user ID with spaces) to confirm the error pages. The automated tests in tests/test_app.py cover the pages and health check, the generate-and-feedback flow, validation errors, admin access, and Gemini failure handling.

Activity 5.2: Deployment with Docker

```
docker build -t fitbuddy .
docker run -p 8000:8000 --env-file .env -v fitbuddy-data:/data fitbuddy
```

The image is based on python:3.12-slim and serves the app with Uvicorn on port 8000; the volume keeps the SQLite database between runs.

Exploring the Website's Web Pages:

Home / Input Page:

[Insert screenshot: FitBuddy home page with the input form]

Description: The home page shows the FitBuddy title and a form for name, user ID, age, weight, goal and intensity. A Generate 7-Day Plan button submits the form, and a loading overlay appears while the AI works.

Result Page:

[Insert screenshot: Generated 7-day plan, nutrition tip and feedback box]

Description: After generation, the result page shows the user's details, the Day 1 to Day 7 plan and the nutrition tip. A feedback box lets the user request changes and receive a full revised plan.

Admin Page:

[Insert screenshot: Admin page listing users]

Description: The admin page lists stored users with their details and plans and allows a user to be deleted. Access is meant to be protected by the admin username and password from the environment file.

Conclusion:

FitBuddy is a Generative AI web application built with FastAPI that gives beginners a personalised 7-day workout plan and a nutrition tip, and refines the plan through feedback. It uses Google Gemini for content generation, SQLite for storage and Docker for portable deployment, with retry handling, input validation and safety-focused prompts. The project covered model selection, backend and frontend development, testing and deployment. Future work includes enforcing admin login on all admin routes, user accounts, progress tracking, structured plan output, multi-language support and cloud deployment with a production database.